

Laboratory Report: Dynamic Web Application Development

Date: September 2, 2026

Subject: Building Interactive Client-Side Applications with HTML5, CSS3, and JavaScript

1. Abstract

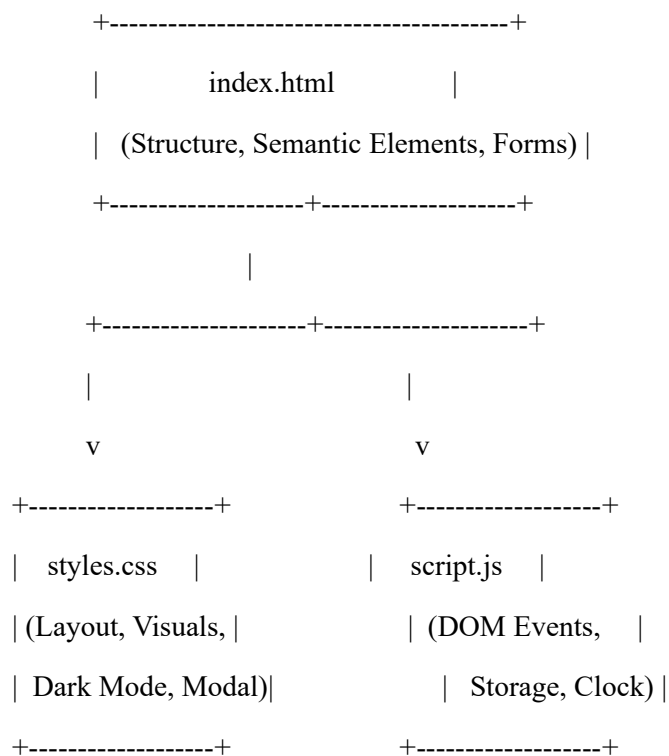
For this lab, I built a functional, single-page web app from scratch using HTML5, CSS3, and modern vanilla JavaScript. The goal was to see how much interactive functionality I could pull off natively without reaching for heavy frameworks like React or Vue.

I implemented a live updating clock in the header, a dark mode theme toggle that remembers user preferences using localStorage, a dynamic task list with item deletion, a contact form with email validation that pops up a custom modal, and a fun little keyboard shortcut feature to change background colors on the fly. Overall, the project was a great hands-on exercise in managing the DOM, handling events efficiently, and keeping the code clean and structured.

The application is structured around a classic three-layer separation of concerns: layout structure (index.html), visual styles (styles.css), and dynamic behavior (script.js).

1.2 Architecture

I kept things simple by breaking the app into three classic web layers: content/structure (index.html), presentation (styles.css), and dynamic behavior (script.js).



1.3 HTML5 Structure

The HTML provides a logical layout using semantic HTML tags:

- `<header>`: Holds the main title (`#heading`), the dark mode toggle button (`#themeToggle`), and standard navigation links.
- `<main>`: Broken up into several specific sections:
- A Task Manager with a text input, an add button, and an empty unordered list ready to be populated.
- Basic Information sections ("About Us" and "Services").
- A Contact Form setup alongside a hidden Modal overlay that steps in when the form is submitted.

2. CSS Styling

The CSS keeps the app clean and modern:

Standard Box Model: Includes reset rules, crisp line heights, flexible spacing, and column-based form layouts using Flexbox.

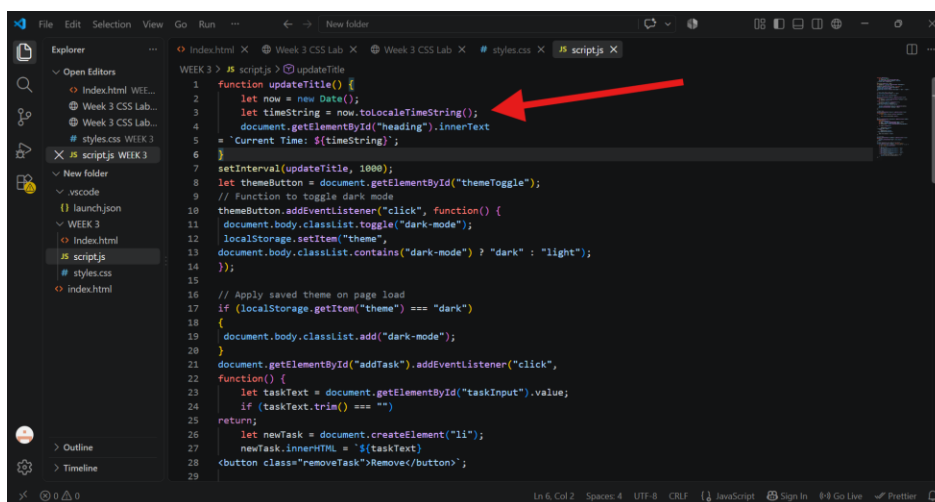
Dark Mode Theme: Controlled by a simple `.dark-mode` CSS class that flips the default light background to black and text to white.

Centered Modal Window: Positioned using `position: fixed`; combined with CSS offsets (`top: 50%`; `left: 50%`; `transform: translate(-50%, -50%)`;) so it pops up cleanly in the middle of the screen over existing content.

Step-by-Step Breakdown of the Code

1. Real-Time Clock Header

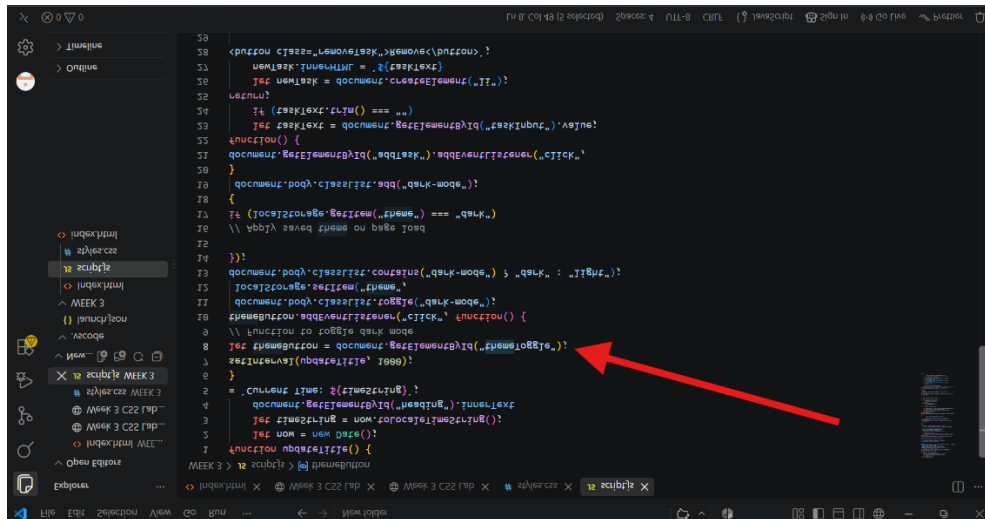
To make the site feel alive right when you open it, I added a real-time clock to the main header using JavaScript's native date object and interval timing.



How it works: `new Date()` grabs the local time, `.toLocaleTimeString()` formats it nicely, and `setInterval` runs the function every second (1000ms) to keep the text continuously updated.

2. Theme Switcher with Persistence

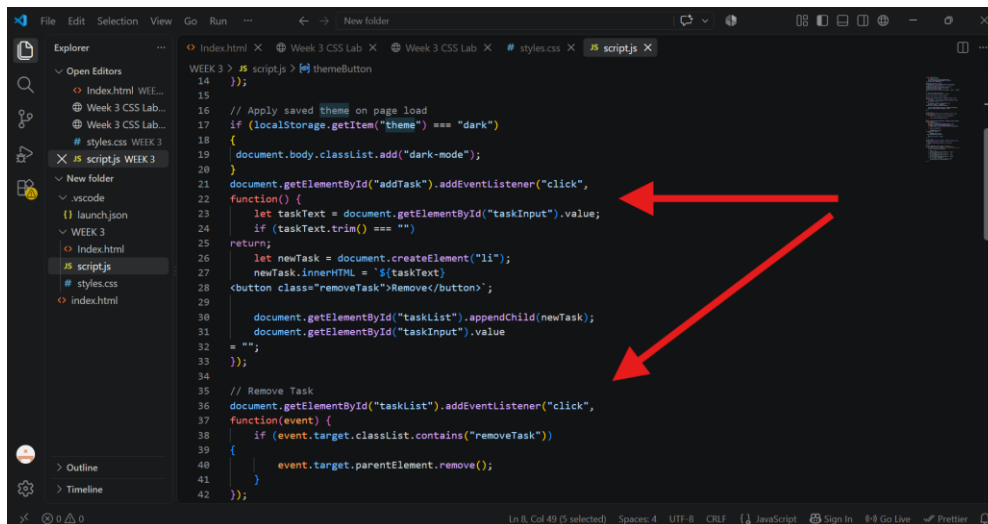
Nobody likes getting blinded by a bright white background at night, so I added a theme switcher. Better yet, it remembers what you picked even if you refresh or leave the page.



How it works: Clicking the button toggles `.dark-mode` on the `<body>`. A quick conditional saves "dark" or "light" directly into the browser's `localStorage`. Whenever the script loads, it checks `localStorage` first and reapplies dark mode if needed.

3. Dynamic Task Manager (Event Delegation)

The task list allows users to add items dynamically and delete them when done. Instead of binding event listeners to every single new task, I used event delegation.

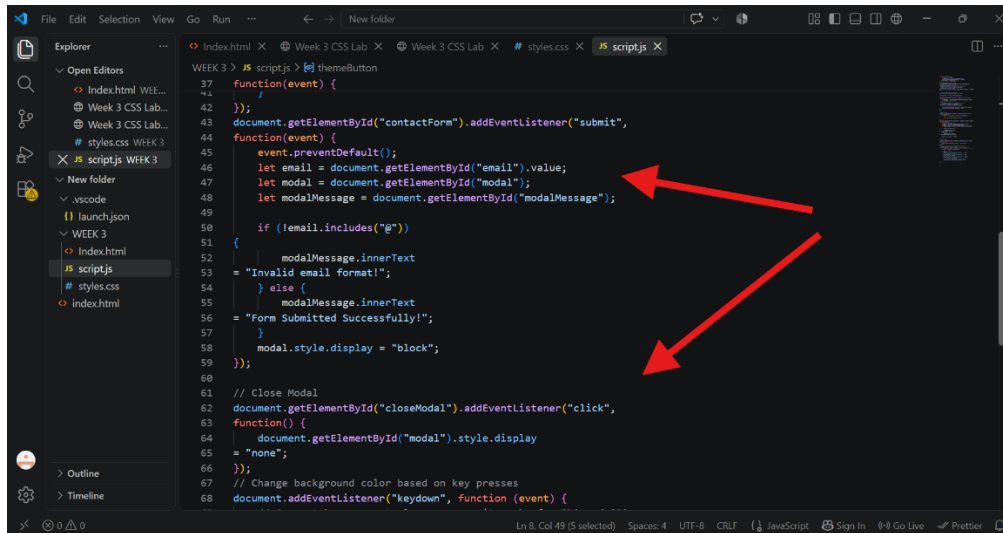


How it works: Creating a task builds an `<li>` element with a nested "Remove" button and appends it to `#taskList`. The deletion logic attaches a single click listener to the parent list container (`#taskList`). When

any child inside the list is clicked, the event bubbles up. If the target has the class `removeTask`, its parent `<li>` is deleted from the page.

4. Form Interception & Custom Modal Popups

Instead of letting the browser do a full page refresh on form submission, I intercepted the submit event to perform validation and show a custom overlay message.



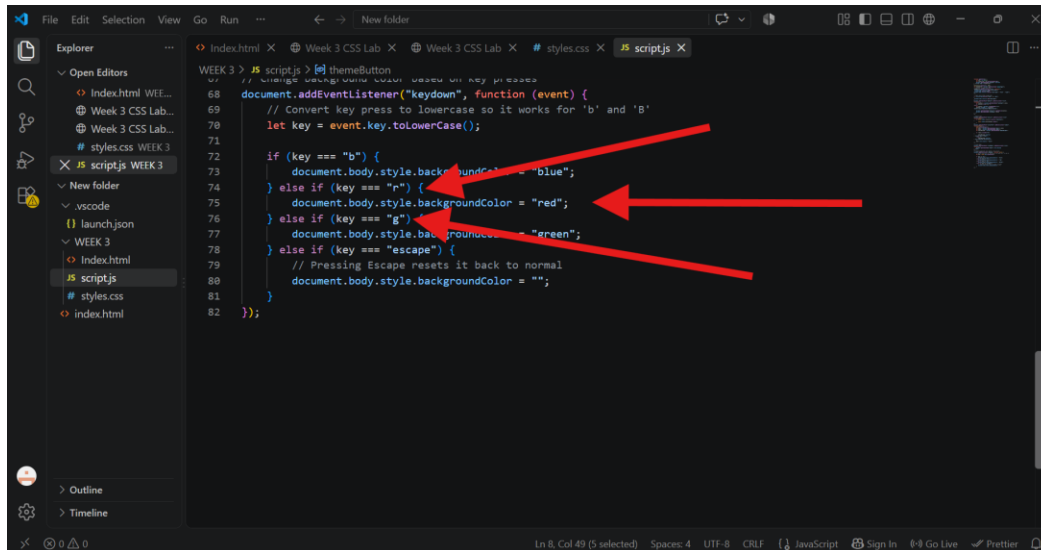
How it works: `event.preventDefault()` stops the default HTML form action. If the email field doesn't contain an `@` symbol, an error message is set; otherwise, it confirms success. Setting `modal.style.display = "block"` reveals the hidden CSS modal, while the close button sets it back to `"none"`.

5. Keyboard Color Controls

As a fun feature for dynamic page restyling, I bound background color changes directly to global keypress events.

NAME: BRIAN VIEL V. BORJE
SECTION: BSIS 2B

PROF/INST: MR. GUILLERMO RED
SUBJECT: WEB SYSTEMS



```
WEEK 3 > .\script.js > themeButton
// Create a button that changes the background color when key pressed
68 document.addEventListener("keydown", function (event) {
69   // Convert key press to lowercase so it works for 'b' and 'B'
70   let key = event.key.toLowerCase();
71
72   if (key === "b") {
73     document.body.style.backgroundColor = "blue";
74   } else if (key === "r") {
75     document.body.style.backgroundColor = "red";
76   } else if (key === "g") {
77     document.body.style.backgroundColor = "green";
78   } else if (key === "escape") {
79     // Pressing Escape resets it back to normal
80     document.body.style.backgroundColor = "";
81   }
82 });
```

How it works: Listening for keydown on the global document lets me check what key was pressed using event.key. Normalizing input with .toLowerCase() handles both capital and lowercase letters. Pressing 'B', 'R', or 'G' applies an inline background color, while pressing Escape clears the style entirely.

REFLECTION

When I was working on this demo I was initially had trouble because I was busy during the week barely getting any rest the day before but I got over it and got my grip on coding back so I did everything what was on the lms said and by that I meant I copy pasted everything written on the instructions.

I had a few mishaps especially the placement of the codes some did not explicitly say where should I place the codes and after six to seven tries I managed to figure out where they should be and fixed it but then I had to add the color change feature and used ai instead and asked where I should place it.

Creating these programs is really fun and soothing especially when listening to Lady Gaga's Abracadabra coding is exciting and all but I really need to memorize the placement and codes on html, css, and java especially java cause its sequences in its animation not to mention css designing and htmls codes.

AI USAGE: 37%

